Inheritance: The mechanism of deriving a new class (derived class/subclass) from an old (base class) one is called Inheritance/Derivation. The derived class inherits some or all of the traits from the base class.

Defining derived class/Syntax:

```
class derived-class-name:private/public base-class-name {

};
```

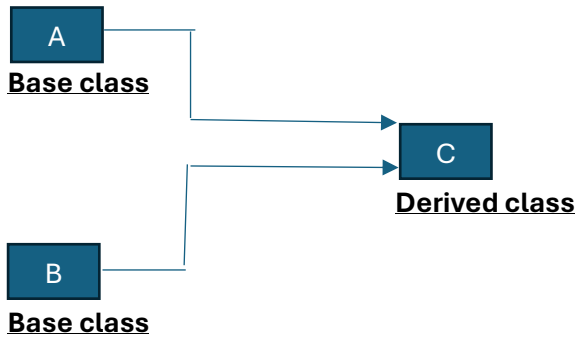
NOTE: Visibility mode is optional, if present then it can be private/public. By default it is private, if we have not mention. Visibility mode specifies whether the features of base class are publicly or privately derived.

Visibility of inherited members:

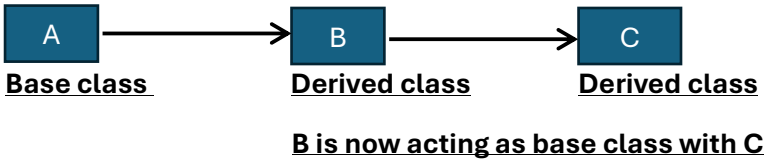
Base class visibility	**Derived class visibility**		
	Public derivation	**Private derivation**	**Protected derivation**
Private	**Not inherited**	**Not inherited**	**Not inherited**
Protected	**Protected**	**Private**	**Protected**
Public	**Public**	**Private**	**Protected**

Single Inheritance: A derived class with only base class, is called single inheritance.

Multiple Inheritance: A derived class with many base classes is called multiple inheritance.

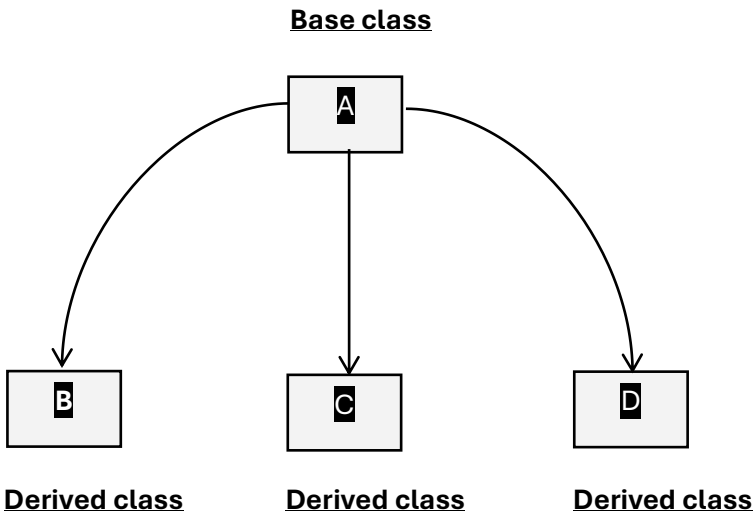


Multilevel Inheritance: The mechanism of deriving a class from another "derived class" is known as multilevel inheritance.

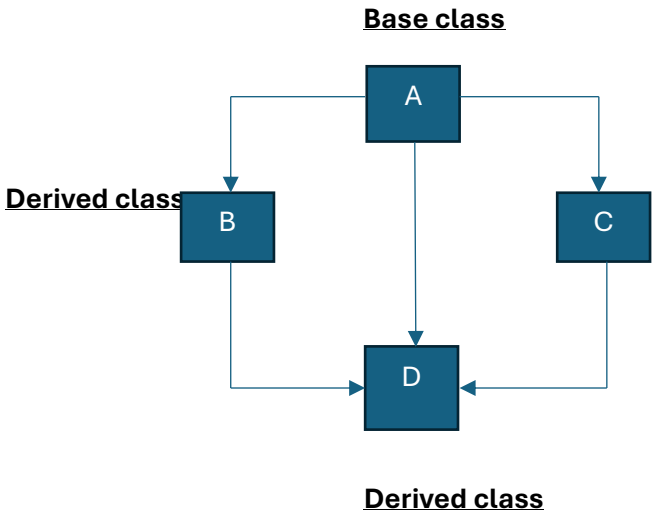


B is now acting as base class with C

Hierarchical Inheritance: A single base class(superclass) has multiple derived classes(subclasses).



Hybrid Inheritance: It is a combination of any of the inheritance i.e single, multiple, multilevel, hierarchical inheritance.



Virtual base classes: Virtual base classes are used in virtual inheritance in a way of preventing multiple “instances” of a given class appearing in an inheritance hierarchy when using multiple inheritances.

Abstract classes: An abstract class is a class that is designed to be specifically used as a base class. An abstract class contains at least one pure virtual function. You declare a pure virtual function by using a pure specifier (=0) in the declaration of a virtual member function in the class declaration. **Constructor in derived classes:** A constructor in a derived class is a special member function that initializes objects of the derived class. When a derived class inherits from a base class, its constructor often needs to initialize both the base class part and its own specific attributes.

Member classes(Nesting of classes): In C++ nesting of classes means defining a class inside another class.

Making private member inheritable: In C++, private members of a base class are not accessible directly in derived classes. However, you can manage access to private members in several ways, allowing them to be effectively "inheritable" in terms of functionality.

1.Change base class member from private to protected.

2.Friend classes.

Pointers, Virtual functions and Polymorphism:

Pointers: A pointer is a derived data type that refers to another data variable by storing the variable memory location/address rather than data. A pointer variable defines where to get the value of a specific data variable instead of defining actual data.

Syntax: datatype *pointer-variable/name;

Ex: int *p;

Operator used:

“&” -> Ampersand -> Reference-> Address of operator.

“*” -> Asterisk ->De-reference -> 1. It is used for declaring pointer. 2. It is used to give value at address operator.

Pointer arithmetic: *(pointer-name+ i (size of data type))

Pointers to objects: Object pointer are useful in creating object at run time. We can also use an object pointer to access the public member of an object.

this pointer: The 'this' pointer is a special pointer in C++ language that is implicitly available within the scope of non-static member functions of a class or structure. It represents the address of the current object instance on which the member function is being called.

NOTE: ‘this’ pointer is not available in static member functions as static member functions can be called without any object (with class name).

Pointer to derived class: In C++, a pointer to a derived class is a pointer that stores the address of a derived class object. A pointer to a derived class can access all the members of the derived class, including those inherited from the base class.

Pointer to pointer: Pointer to a pointer is a form of multiple in-direction or a chain of pointers. Normally, a pointer contains the address of a variable. When we define a pointer to a pointer, the first pointer contains the address of the second pointer, which points to the location that contains the actual value.

Virtual function: A virtual function (also known as virtual methods) is a member function that is declared within a base class and is re-defined (overridden) by a derived class. When you refer to a derived class object using a pointer or a reference to the base class, you can call a virtual function for that object and execute the derived class's version of the method.

Key features:

1.Virtual functions ensure that the correct function is called for an object, regardless of the type of reference (or pointer) used for the function call.

2.They are mainly used to achieve Runtime Polymorphism.

3.Functions are declared with a virtual keyword in a base class.

4.The resolving of a function call is done at runtime.

Rules for Virtual Functions:

1.Virtual functions cannot be static.

2.A virtual function can be a friend function of another class.

3.Virtual functions should be accessed using a pointer or reference of base class type to achieve runtime polymorphism.

4.The prototype of virtual functions should be the same in the base as well as the derived class.

5.They are always defined in the base class and overridden in a derived class. It is not mandatory for the derived class to override (or re-define the virtual function), in that case, the base class version of the function is used.

6.A class may have a virtual destructor but it cannot have a virtual constructor.

Pure Virtual function: A pure virtual function is a virtual function that has no definition within the class.

Characteristics of a pure virtual function:

1.A pure virtual function is a "do nothing" function. Here "do nothing" means that it just provides the template, and derived class implements the function.

2.It can be considered as an empty function means that the pure virtual function does not have any definition relative to the base class.

3.Programmers need to redefine the pure virtual function in the derived class as it has no definition in the base class.

4.A class having pure virtual function cannot be used to create direct objects of its own. It means that the class is containing any pure virtual function then we cannot create the object of that class. This type of class is known as an abstract class.

Syntax:

1. virtual void display() = 0;

2. virtual void display() {}

Difference between Virtual and Pure Virtual function:

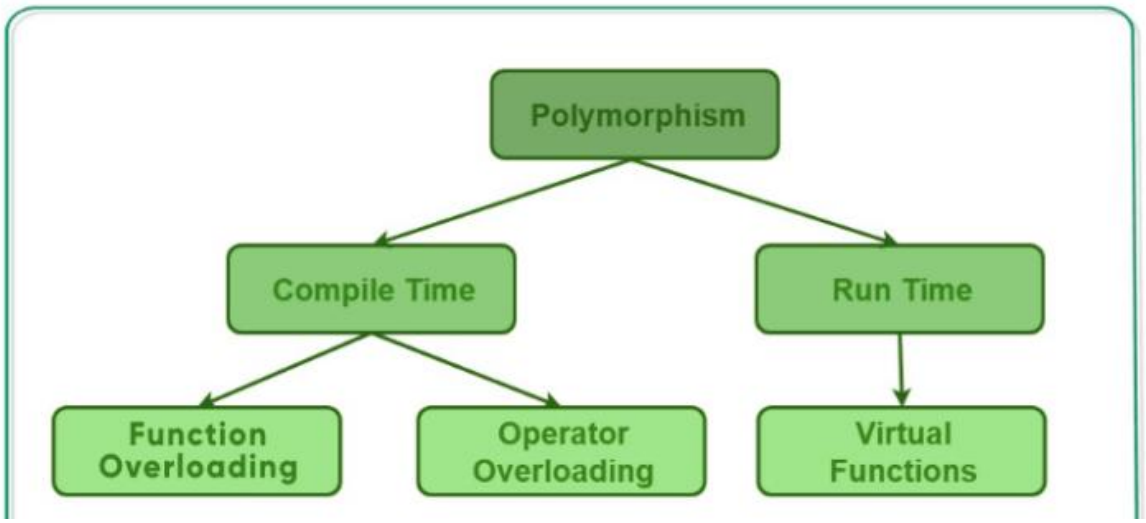
Virtual function	Pure virtual function
A virtual function is a member function in a base class that can be redefined in a derived class.	A pure virtual function is a member function in a base class whose declaration is provided in a base class and implemented in a derived class.
The classes which are containing virtual functions are not abstract classes.	The classes which are containing pure virtual function are the abstract classes.
In case of a virtual function, definition of a function is provided in the base class.	In case of a pure virtual function, definition of a function is not provided in the base class.
The base class that contains a virtual function can be instantiated.	The base class that contains a pure virtual function becomes an abstract class, and that cannot be instantiated.
If the derived class will not redefine the virtual function of the base class, then there will be no effect on the compilation.	If the derived class does not define the pure virtual function; it will not throw any error but the derived class becomes an abstract class.
All the derived classes may or may not redefine the virtual function.	All the derived classes must define the pure virtual function.

Polymorphism: The word "polymorphism" means having many forms. In simple words, we can define polymorphism as the ability of a message to be displayed in more than one form.

Types of Polymorphism:

1.Compile-time Polymorphism.

2.Runtime Polymorphism.



1. Compile-Time Polymorphism: This type of polymorphism is achieved by function overloading or operator overloading.

A. Function Overloading: When there are multiple functions with the same name but different parameters, then the functions are said to be overloaded, hence this is known as Function Overloading. Functions can be overloaded by changing the number of arguments or/and changing the type of arguments.

B. Operator Overloading: C++ has the ability to provide the operators with a special meaning for a data type, this ability is known as operator overloading.

2. Runtime Polymorphism: This type of polymorphism is achieved by **Function Overriding**. Late binding and dynamic polymorphism are other names for runtime polymorphism. The function call is resolved at runtime in runtime polymorphism. In contrast, with compile time polymorphism, the compiler determines which function call to bind to the object after deducing it at runtime.

A. Virtual Function: A virtual function is a member function that is declared in the base class using the keyword virtual and is re-defined (Overridden) in the derived class.

Some Key Points About Virtual Functions:

1.Virtual functions are Dynamic in nature.

2.They are defined by inserting the keyword "**virtual**" inside a base class and are always declared with a base class and overridden in a child class.

3.A virtual function is called during runtime.